

# Point Location and Trapezoidal Maps

## 1 Planar Point Location Problem

- partitioning in slabs
- analysis of a query algorithm

## 2 Trapezoidal Maps

- refining a subdivision
- trapezoidal shapes
- linear storage cost

## 3 Trapezoidal Maps in CGAL

- point location queries in CGAL

MCS 481 Lecture 18  
Computational Geometry  
Jan Verschelde, 5 October 2026

# Point Location and Trapezoidal Maps

## 1 Planar Point Location Problem

- partitioning in slabs
- analysis of a query algorithm

## 2 Trapezoidal Maps

- refining a subdivision
- trapezoidal shapes
- linear storage cost

## 3 Trapezoidal Maps in CGAL

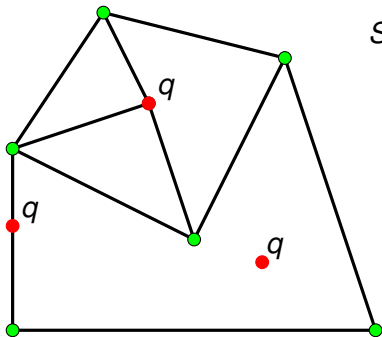
- point location queries in CGAL

# the planar point location problem

Given a map, do you know where you are?

Input: a planar subdivision  $S$ ,  $n = \# \text{edges of } S$ ,  
a query point  $q = (q_x, q_y)$ .

Output: face of  $S$  that contains  $q$ , or eventually  
the edge which contains  $q$ , or the vertex matching  $q$ .



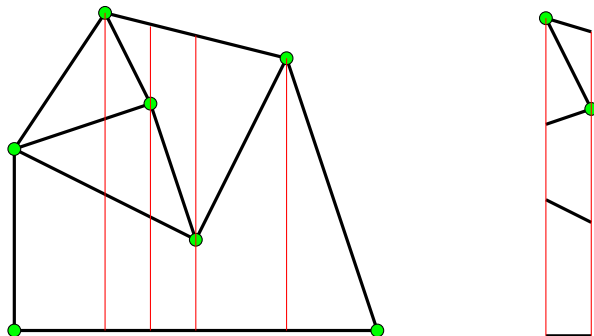
$S, n = 10$

3 cases:

- $q$  in face
- $q$  on edge
- $q$  is vertex

# partitioning in slabs — a data structure solution

- 1 Store the  $x$ -coordinates in a sorted array.
- 2 Draw vertical lines through each vertex.

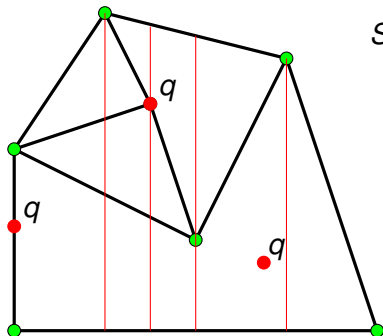


Properties of a slab:

- 1 edges in a slab are sorted from top to bottom,
- 2 every region in a slab belongs to a unique face.

# queries in a partitioned subdivision

Given: a query point  $q = (q_x, q_y)$  and a partitioned subdivision:



$S, n = 10$

3 cases:

- $q$  in face
- $q$  on edge
- $q$  is vertex

Given  $q = (q_x, q_y)$ ,

- 1 use  $q_x$  to locate the slab that contains  $q$ ;
- 2 use  $q_y$  to find where  $q$  is in the slab.

# Point Location and Trapezoidal Maps

## 1 Planar Point Location Problem

- partitioning in slabs
- analysis of a query algorithm

## 2 Trapezoidal Maps

- refining a subdivision
- trapezoidal shapes
- linear storage cost

## 3 Trapezoidal Maps in CGAL

- point location queries in CGAL

# cost of a query algorithm

Given are the coordinates  $(q_x, q_y)$  of  $q$ ,

- 1 use  $q_x$  to run a binary search, to find the slab;
- 2 use  $q_y$  to run a binary search on segments in the slab.

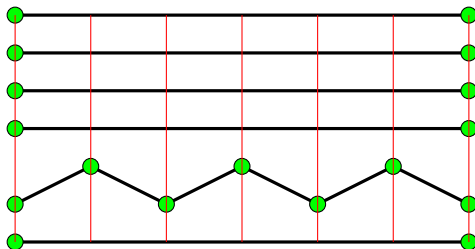
What is the cost of this query algorithm?

- 1 Let  $S$  have  $n$  edges. Then there are at most  $2n$  vertices.
- 2 Each binary search takes  $O(\log(n))$  operations.

Therefore, the cost is  $O(\log(n))$ .

## storage cost of slab partitioning

Consider a subdivision with  $n = 11$  edges:



## The partitioning in slabs:

- 1 there are 6 slabs,
- 2 every slab has 6 edges;

and  $6 = 11/1.8333... = n/1.8333....$

The storage cost is  $O(n^2)$ , although  $O(n\sqrt{n})$  is more likely.



# doubly connected edge list representation

## Exercise 1:

The coordinates of the vertices in the drawing of the previous slide are  $(0, 0)$ ,  $(60, 0)$ ,  $(0, 5)$ ,  $(10, 10)$ ,  $(20, 5)$ ,  $(30, 10)$ ,  $(40, 5)$ ,  $(50, 10)$ ,  $(0, 15)$ ,  $(0, 20)$ ,  $(0, 25)$ ,  $(0, 30)$ ,  $(60, 5)$ ,  $(60, 15)$ ,  $(60, 20)$ ,  $(60, 25)$ ,  $(60, 30)$ .

Define the doubly connected edge list representation for the partitioned subdivision.

Use this representation to explicitly count the storage cost.

# Point Location and Trapezoidal Maps

## 1 Planar Point Location Problem

- partitioning in slabs
- analysis of a query algorithm

## 2 Trapezoidal Maps

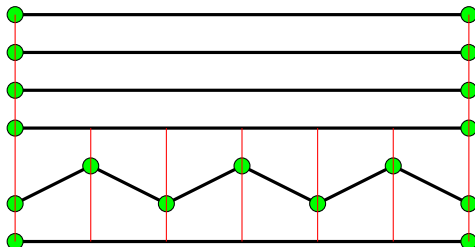
- refining a subdivision
- trapezoidal shapes
- linear storage cost

## 3 Trapezoidal Maps in CGAL

- point location queries in CGAL

# refining a subdivision — a motivating example

Use shorter extensions:



Through each vertex we add a vertical extension

- 1 to the next edge below the vertex,
- 2 to the next edge above the vertex.

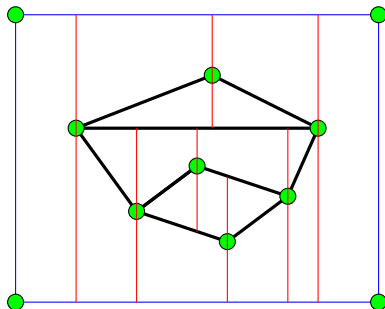
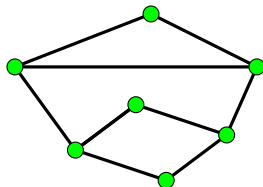
We thus arrive at a storage cost of  $O(n)$ ,  
as we have at most  $2n$  vertices, for  $n$  edges in the subdivision.

**Exercise 2:** Count the storage cost for the above subdivision.

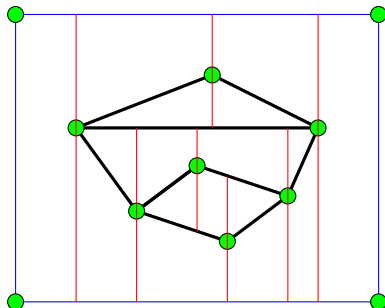
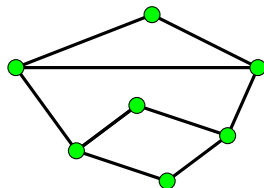
# trapezoidal maps

Assume the subdivision  $S$  is bounded and in general position:

- 1  $S$  is entirely contained in a rectangular box  $R$ ,
- 2  $S$  contains no vertical edges.



# a trapezoidal decomposition of a subdivision



Constructing a trapezoidal decomposition  $T(S)$  of a subdivision  $S$ :

for every edge  $e$  in  $S$  do

for every end point  $p$  of  $e$  do

add a vertical extension from  $p$  to the next edge below  $p$

add a vertical extension from  $p$  to the next edge above  $p$

# Point Location and Trapezoidal Maps

## 1 Planar Point Location Problem

- partitioning in slabs
- analysis of a query algorithm

## 2 Trapezoidal Maps

- refining a subdivision
- **trapezoidal shapes**
- linear storage cost

## 3 Trapezoidal Maps in CGAL

- point location queries in CGAL

# faces of a trapezoidal decomposition

## Lemma (trapezoidal faces)

*Every face in a trapezoidal decomposition  $T(S)$  has one or two vertical sides and exactly two skew sides.*

*Proof.* We first show that every face  $f$  of  $T(S)$  is convex.

The corner of  $f$  is either a corner of the bounding box  $R$ , or where a vertical extension meets an edge of  $S$ .

In both cases, the angle at each corner is less than  $\pi$ .

Since no interior angle of  $f$  is larger than  $\pi$ ,  $f$  is convex.

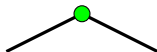
Because each face is convex, there are at most two vertical sides.

There is at least one vertical side, because  $S$  is bounded by a box.

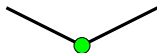
## every face has exactly two skew sides

Assume there would be more than two skew sides.

But then we would have one of the following cases:



or



But we cannot have inner vertices, as vertical extensions were added in the construction of  $T(S)$  through every vertex of  $S$ .

Our assumption is false, which concludes the proof.

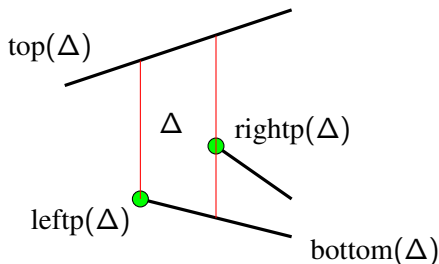
Q.E.D.



# trapezoidal shapes

Every face  $\Delta$  of  $T(S)$  has a trapezoidal shape, and is determined by

- 1 the  $\text{top}(\Delta)$  and  $\text{bottom}(\Delta)$  edges,
- 2 the  $\text{leftp}(\Delta)$  and  $\text{rightp}(\Delta)$  vertices.



# Point Location and Trapezoidal Maps

## 1 Planar Point Location Problem

- partitioning in slabs
- analysis of a query algorithm

## 2 Trapezoidal Maps

- refining a subdivision
- trapezoidal shapes
- linear storage cost

## 3 Trapezoidal Maps in CGAL

- point location queries in CGAL

## five cases for the left point

For  $\text{leftp}(\Delta)$ , we have five possible cases:



The fifth case:  $\text{leftp}(\Delta)$  is a vertex of the bounding box  $R$ .

**Exercise 3:** Draw the possible cases for  $\text{rightp}(\Delta)$ .

# linear storage cost

## Lemma (linear storage cost)

*Let  $S$  be a subdivision with  $n$  edges and bounding box  $R$ .  
Its trapezoidal decomposition  $T(S)$  has at most  $6n + 4$  vertices  
and at most  $3n + 1$  trapezoids.*

*Proof.*

$$\begin{array}{rcl} \text{A vertex of } T(S) \text{ is either a vertex of } R: & & 4, \\ \text{or an endpoint of an edge of } S: n \times 2 & = & 2n, \\ \text{or the end of a vertical extension: } (2n) \times 2 & = & 4n, \\ & & \hline & & 6n + 4. \end{array}$$

For the number of trapezoids, consider the cases for  $\text{leftp}(\Delta)$ .

# counting the number of trapezoids

## Lemma (linear storage cost)

Let  $S$  be a subdivision with  $n$  edges and bounding box  $R$ .  
Its trapezoidal decomposition  $T(S)$  has at most  $6n + 4$  vertices  
and at most  $3n + 1$  trapezoids.

*Proof continued.* Consider the five cases for  $\text{left}(\Delta)$ :



Either  $\text{leftp}(\Delta)$  is a corner of  $R$ : 1,  
or  $\text{leftp}(\Delta)$  is endpoint of an edge:  $n$ ,  
but  $\text{leftp}(\Delta)$  can be the left for at most 2 trapezoids:  $2n$ ,  

---

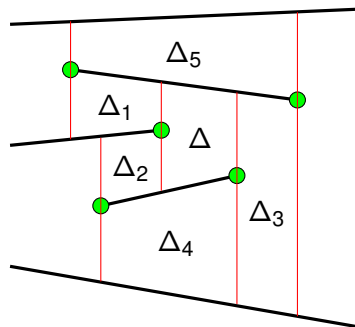
 $3n + 1$ .

Q.E.D.

# adjacent trapezoids

## Definition (adjacent trapezoids)

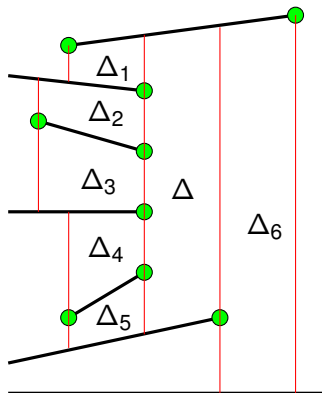
Trapezoids  $\Delta$  and  $\Delta'$  are *adjacent* if they meet along a vertical edge.



The trapezoids  $\Delta_1$ ,  $\Delta_2$ , and  $\Delta_3$  are adjacent to  $\Delta$ .  
The trapezoids  $\Delta_4$  and  $\Delta_5$  are *not* adjacent to  $\Delta$ .

## a subdivision not in general position

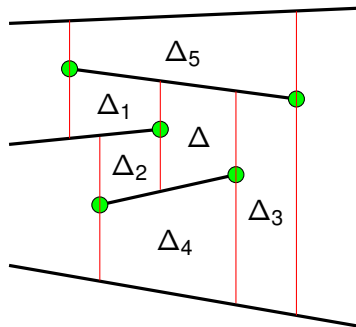
A subdivision with vertical edges:



If not in general position,  
 $\Delta$  can have an arbitrary number of adjacent trapezoids.

# storing a trapezoidal decomposition $T(S)$

For a subdivision  $S$  in general position:



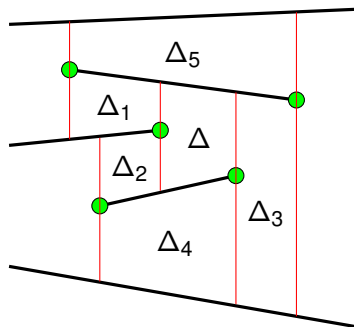
A doubly edge connected edge list can be used to store  $T(S)$ .  
However, any  $\Delta$  is uniquely determined by

- 1 the edges  $\text{top}(\Delta)$  and  $\text{bottom}(\Delta)$ , and
- 2 the vertices  $\text{leftp}(\Delta)$  and  $\text{rightp}(\Delta)$ .



# defining a specialized data structure

**Exercise 4:** For the trapezoidal subdivision  $T(S)$  below:



define the specialized data structure to store  $T(S)$ .  
Explain how the adjacency information can be stored,  
eventually with some additional pointers.

# Point Location and Trapezoidal Maps

## 1 Planar Point Location Problem

- partitioning in slabs
- analysis of a query algorithm

## 2 Trapezoidal Maps

- refining a subdivision
- trapezoidal shapes
- linear storage cost

## 3 Trapezoidal Maps in CGAL

- point location queries in CGAL

# Trapezoidal Maps in CGAL

- Look in the tutorial on 2D Arrangements, in particular in the section Point-Location Queries.
- As the point-location strategy, select `Arr_trapezoid_ric_point_location<Arrangement>` which implements a variant of Ketan Mulmuley's algorithm.
- Ketan Mulmuley: **A Fast Planar Partition Algorithm, I** *Journal of Symbolic Computation* Vol. 10, pages 253-280, 1990.
- An example: `curve_history.cpp`.

# summary and exercises

The point location problem is a query on a map  $S$ , solved by the construction of a trapezoidal decomposition of  $S$ .

We covered section 6.1 in the textbook.

Consider the following activities, listed below.

- 1 Write the solutions to exercises 1, 2, 3, and 4.
- 2 Consult the CGAL documentation and example code on trapezoidal maps.
- 3 Consider the exercises 1, 3, 4 in the textbook.